

Anomaly Detection of Interaction Behaviors in Streaming Graphs

Shuai Ren
Guangzhou University
CIAT & Guangdong Key Laboratory
of Industrial Control System Security
& Huangpu Research School
Guangzhou, Guangdong, China
shuairn.cs@gmail.com

Fan Zhang*
Guangzhou University
CIAT & Guangdong Key Laboratory
of Industrial Control System Security
& Huangpu Research School
Guangzhou, Guangdong, China
zhangf@gzhu.edu.cn

Bolin Wang
Guangzhou University
CIAT & Guangdong Key Laboratory
of Industrial Control System Security
& Huangpu Research School
Guangzhou, Guangdong, China
tenmumulynn@gmail.com

Xiang Zhao
National University of Defense
Technology
National Key Laboratory of Big Data
and Decision
Changsha, Hunan, China
xiangzhao@nudt.edu.cn

Zhihong Tian
Guangzhou University
CIAT & Guangdong Key Laboratory
of Industrial Control System Security
& Huangpu Research School
Guangzhou, Guangdong, China
tianzhong@gzhu.edu.cn

Abstract

Timely detection of anomalous interactions between different entities is crucial for the security and stability of Web-related systems, which are often modeled as streaming graphs. A major issue in existing anomaly detection methods is the effectiveness and the corresponding interpretability. In this paper, we introduce a novel evaluation metric, namely *Interaction Willingness*, to measure the propensity for entity interactions. Based on this metric, we design two efficient anomaly detection algorithms, *AnoD* and *AnoC*, tailored for real-time detection of two prevalent types of anomalous interaction behaviors (dense-type and counter-type), respectively. Notably, we adopt and extend the High-Order Count-Min Sketch for the implicit storage of streaming graphs, and ensure that both the space cost and the time cost of *AnoD* and *AnoC* for processing each new streaming edge remain constant and user-controllable, with estimation guarantees. Experimental evaluations on 6 real-world datasets demonstrate that the proposed evaluation metric and the associated algorithms achieve superior detection performance with clear interpretability for detecting anomalous interactions.

CCS Concepts

• Information systems → Data stream mining.

Keywords

Anomaly Detection; Streaming Graph; Sketch

ACM Reference Format:

Shuai Ren, Fan Zhang, Bolin Wang, Xiang Zhao, and Zhihong Tian. 2026. Anomaly Detection of Interaction Behaviors in Streaming Graphs. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026.

*Corresponding author: Fan Zhang.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792282>

Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792282>

Resource Availability:

The source code of this paper has been made publicly available at <https://doi.org/10.5281/zenodo.18311927>.

1 Introduction

In real-world Web-related systems, new entities are continuously added over time, and internal entities engage in ongoing interactions. Examples include social media platforms (where new users join and existing users interact, e.g., commenting or following), computer networks (where new devices connect and devices communicate), and financial systems (where new accounts are created and inter-account transfers are issued). Such evolving interactive relations can be effectively modeled as streaming graphs [1].

Given a specific timestamp ts , the streaming graph comprises all the streaming edges (the interactions between the entities) that occurred at and before time ts , as well as the entity nodes corresponding to these streaming edges (i.e., the source nodes and destination nodes of the edges). The goal of anomalous interaction detection in streaming graphs is to compute the anomaly score of each newly incoming streaming edge in real time.

Ideally, anomalous edges correspond to real-world malicious activities such as spam marketing by malicious social media accounts, attacks launched by hackers using compromised devices in computer networks, and money laundering through suspicious financial

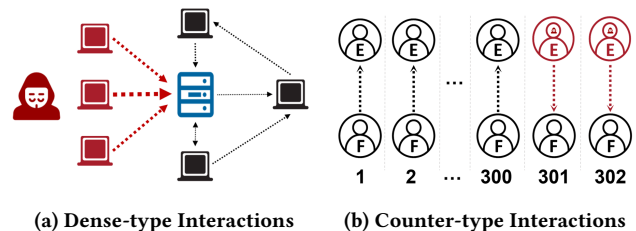


Figure 1: Examples of Two Types of Anomalous Interactions

accounts. Timely detection of these anomalous interactions is critical for maintaining the security and stability of corresponding systems [2–4].

Our investigation distills that there are two prevalent types of anomalous interaction behaviors in real-world scenarios, i.e., dense-type and counter-type anomalies, which cover a large portion of anomalies as reflected by the AUC values in the experiments.

- **Dense-type anomalous interactions** refer to short-term frequent interactions between specific entities in a system, which correspond to real-world malicious activities such as network attacks (e.g., DDoS), illegal marketing (e.g., spam campaigns), and money laundering [5–7]. Figure 1a shows an example of DDoS attacks [8] launched by a hacker’s devices against a target server within a specific period, i.e., flooding the server with excessive service requests, where the line thickness indicates the frequency of the requests. Naturally, short-term frequent interactions occur between the attacker’s devices and the target server due to the attacks, and these high-frequency edges (marked red) are dense-type anomalies.
- **Counter-type anomalous interactions** refer to interactions between entities that deviate significantly from their long-term interaction behaviors, and typically correspond to specific illegal or non-compliant activities in real-world scenarios [9, 10]. For instance, Figure 1b shows a social platform where user E is a blogger and user F often comments on E’s blogs. Over the past year, E has never initiated any proactive interaction with F. However, E recently sent several messages or comments (e.g., “Claim free courses, join group XXX”). This abrupt outreach deviates from their established long-term interaction pattern, which is a counter-type anomaly.

In certain applications, methods based on evaluation rules offer advantages in both detection performance and interpretability compared to learning-based methods [11] (as discussed in Section 2). Nevertheless, a high-quality evaluation metric is critical for rule-based methods, which should satisfy three key criteria simultaneously: (1) Applicable to detecting dense-type and counter-type anomalous interactions; (2) Concise definition and low computation complexity; and (3) Strong discriminative power.

In this paper, we introduce a novel evaluation metric, namely *Interaction Willingness (IW)* (defined in Section 4.1), which satisfies all the aforementioned criteria. IW quantifies the likelihood of directed interactions from one entity to another within a system. Building on this metric and in light of the characteristics of the two types of anomalous interactions, we propose two algorithms, i.e., *AnoD* and *AnoC*, both with constant time complexity on processing a new edge (u, v) (where u denotes the source node and v denotes the destination node) for dense-type and counter-type anomalous score computation, respectively.

Specifically, *AnoD* utilizes a global decay mechanism to model temporal effects, and takes the updated IW strength of interacting entities as the anomaly score for the current interaction from u to v . Besides, by integrating the newly proposed interaction frequency computation with an element-wise decay mechanism, *AnoC* computes the *Frequency-based Interaction Willingness (FIW)* to measure

the expected likelihood of the current interaction. *AnoC* subsequently uses the difference between the normalized FIW strength and the current frequency of the interaction as the anomaly score from u to v . More design rationales are given in Section 4.3.

Notably, as time progresses, new nodes and edges are continuously generated in the corresponding streaming graph. Thus, explicitly storing the streaming graph structure (i.e., storing full structural details; e.g., via adjacency matrices) inevitably incurs prohibitive resource overhead. To address this issue, we adopt and extend the High-Order Count-Min Sketch (HCMS) proposed by Bhatia et al. [12] to enable the implicit storage of streaming graphs. These structures offer excellent operability and reliable guarantees for count estimation, and ensure that both the space cost and the time cost of *AnoD* and *AnoC* for processing each newly incoming streaming edge remain constant and user-controllable (i.e., independent of the streaming graph scale).

The main contributions of this paper are summarized as follows:

- We propose a simple, efficient, and useful evaluation metric, i.e., Interaction Willingness, applicable to the detection of both dense-type and counter-type anomalies.
- We establish theoretical guarantees for IW strength estimation via HCMS and develop an extended HCMS structure (EHCMS for short).
- We design *AnoD* and *AnoC* algorithms to compute the anomaly score of each new edge within constant space and time complexities, which can be simply extended to handle edge deletions.
- The experiments demonstrate excellent detection performance, efficiency, and scalability of the proposed algorithms on 6 public real-world datasets with anomaly labels.

2 Related Work

Existing approaches of anomaly detection in streaming graphs can be broadly categorized into two paradigms: learning-based methods and evaluation rule-based methods.

Learning-Based Methods. Learning-based methods leverage deep learning models to learn and update node or edge embeddings in streaming graphs. The anomaly score of a streaming edge is then computed based on the learned embeddings.

Among these methods, NetWalk [13] detects anomalous streaming edges via dynamic graph embedding techniques and clustering strategies. AddGraph [14] is an end-to-end framework with extended temporal graph convolutional networks (TGCNs) and attention mechanisms, which captures both long-term and short-term patterns in dynamic graphs¹. TADDY [15] was the first to apply Transformer models to anomaly detection in dynamic graphs, enabling it to efficiently capture intertwined spatiotemporal information via the construction of information-dense node encodings. SAD [16] is a semi-supervised method that exploits the potential of unlabeled samples for enhancing anomaly detection performance, via the design of a temporal memory bank and a pseudo-label contrastive learning module. StrGNN [17] emphasizes the structural evolution of the h -hop enclosing subgraph of the target edge, and employs a hybrid architecture integrating graph convolutional networks (GCNs) for feature extraction and gated recurrent units

¹Streaming graphs can be regarded as a special type of dynamic graphs.

(GRUs) for temporal modeling. SLADE [18] develops a deep neural network that leverages temporal deviations in node states for anomaly detection. To enhance the generalization performance of detection models, GeneralDyG [6] employs temporal ego-graph sampling and node-edge alternating graph convolution to perform anomaly detection.

While these methods achieved notable progress, the integration of deep learning models may incur substantial computational overhead [19], rendering such methods hard to handle large-scale real-time anomaly detection tasks. Furthermore, the scarcity of labeled anomalous samples for model training, the lack of anomalous features, and the low interpretability of detection algorithms still hinder their deployment in real-world scenarios [20–22].

Evaluation Rule-Based Methods. Evaluation rule-based methods, including AnoD and AnoC proposed in this paper, use interpretable metrics derived from data characteristics and domain expertise to assess the anomaly score of streaming edges. Compared with learning-based methods, these methods usually deliver lower resource cost, stronger robustness, and enhanced interpretability.

Among these methods, RHSS [23] specializes in detecting sparse connectivity patterns in graphs. CAD [24] identifies node relationships that induce anomalous structural changes, using commute time distances. SedanSpot [25] targets the detection of anomalous streaming edges that emerge abruptly and link sparsely connected components of the graph. Midas [26] targets micro-cluster anomalies in edge streams: it uses Count-Min Sketch to compute edge occurrence frequencies at each timestamp, then employs the chi-square test to assess deviations from typical edges to generate anomaly scores. F-FADE [27] detects abrupt surges in interactions between specific node pairs and rapid shifts in node community affiliations via frequency factorization, to identify anomalous streaming edges. MStream [28] employs locality-sensitive hash functions to map incoming similar edge tuples into fixed buckets for similar edges, then utilizes a temporal scoring function to identify anomalous activities. DenseAlert [29] maintains and incrementally updates dense sub-tensors within tensor streams. PENminer [30] concentrates on exploring the persistence of activity segments in evolving networks—specifically, the length and regularity of repeated edge update sequences. AnoEdge [12] employs an extended Count-Min Sketch architecture to efficiently detect anomalous streaming edges within dense subgraph structures.

Compared with existing methods, our proposed AnoD and AnoC algorithms are specifically designed for dense-type and counter-type anomalous interactions, respectively. The constant time and space complexities enable real-time processing of the above two prevalent anomaly types.

3 Preliminaries

In this section, we formally introduce the problem of anomaly detection of interaction behaviors in streaming graphs. We use nodes to represent entity objects, edges to represent interaction behaviors, and timestamps to represent the occurrence time of interactions. The streaming graph is a directed graph, as defined in the following.

Definition 3.1 (Streaming Graph). A streaming graph is defined as $G(U, V, E, W, T) = \{e_1, e_2, \dots\}$ which comprises streaming edges

accumulated in the order of their generation up to a given timestamp ts . Specifically, a streaming edge e_i is the i -th edge added to G , defined as a 4-tuple $e_i = (u_i, v_i, w_i, t_i)$, where $u_i \in U$ is the source node of e_i , $v_i \in V$ is the destination node of e_i , $w_i \in W$ denotes the weight of e_i (e.g., interaction intensity), and $t_i \in T$ denotes the timestamp of e_i .

The problem definition of anomaly detection of interaction behaviors in streaming graphs is given in Definition 3.2. Notably, no assumption is made that the node set $U \cup V$ is known a priori, i.e., new endpoint nodes can be created associatedly during the generation of streaming edges.

Definition 3.2 (Problem Definition). Given a streaming graph G at the current timestamp and a new edge $e_i \notin G$, the problem is to compute the anomaly score $S(e_i)$ based on G and the intermediate results from processing the previous edge (if available). A higher value of $S(e_i)$ indicates a greater probability that e_i corresponds to an anomalous interaction.

We also extend the algorithms to process edge deletions as discussed at the end of Section 4.4.

4 Methodology

In this section, we first introduce a lightweight but effective evaluation metric, i.e., Interaction Willingness (Section 4.1), as well as the data structures, core operations, and count estimation guarantees of High-Order Count-Min Sketch (HCMS) and its extended variant EHCMS to achieve low space and time costs (Section 4.2). Then, we present the design rationales and two detection algorithms (AnoD and AnoC) tailored for dense-type and counter-type anomalous interactions, respectively.

4.1 Evaluation Metric—Interaction Willingness

The out-degree and in-degree of a node directly indicate its propensity to initiate and receive directed interactions, respectively. The Interaction Willingness (IW), a computationally efficient metric defined as the sum of the source node's out-degree and the target node's in-degree in the current streaming graph, quantifies the inherent likelihood of a directed interaction from the source node to the target node.

Definition 4.1 (Interaction Willingness). For a streaming graph G (corresponding to a system state at a given timestamp), the Interaction Willingness $I(u_i, v_i)$ from node u_i to its neighboring node v_i , i.e., the propensity of a directed interaction from u_i to v_i , is computed by Equation 1:

$$I(u_i, v_i) = d_{out}(u_i) + d_{in}(v_i) \quad (1)$$

where $d_{out}(u_i)$ denotes the out-degree of node u_i , $d_{in}(v_i)$ denotes the in-degree of node v_i .

It is worth noting that this definition can be generalized to a weighted sum form, i.e., $I(u_i, v_i) = \alpha \cdot d_{out}(u_i) + \beta \cdot d_{in}(v_i)$, where parameters α and β balance the contributions of $d_{out}(u_i)$ and $d_{in}(v_i)$. This generalized form allows for flexibility in scenarios where the

²In the following of the paper, we adopt a default setting of equal contributions (i.e., $\alpha = \beta = 1$). Further experimental validation and analysis of these parameters are given in Appendix E.

Algorithm 1: Core Operations of HCMS

```

1 Procedure Initialize( $n_r, n_b$ ):
2   for  $j \leftarrow 1$  to  $n_r$  do
3      $h_{r_j} : \mathcal{V} \rightarrow [0, n_b], h_{c_j} : \mathcal{V} \rightarrow [0, n_b]$ 
4      $M_j \leftarrow [0]_{n_b \times n_b}$ 
5 Procedure Update( $e_i = (u_i, v_i, w_i, t_i)$ ):
6   for  $j \leftarrow 1$  to  $n_r$  do
7      $M_j[h_{r_j}(u_i)][h_{c_j}(v_i)] \leftarrow M_j[h_{r_j}(u_i)][h_{c_j}(v_i)] + w_i$ 
8 Procedure Decay( $\gamma$ ):
9   for  $j \leftarrow 1$  to  $n_r$  do
10     $M_j \leftarrow \gamma \cdot M_j$ 

```

roles of initiators and recipients differ significantly in anomaly measurement.

4.2 Implicit Storage of Streaming Graphs

To control the overhead of the detection algorithms, we employ the High-Order Count-Min Sketch (HCMS) and its extended variant, the Extended High-Order Count-Min Sketch (EHCMS), to achieve the implicit (approximate) storage of streaming graphs.

4.2.1 Data Structures.

High-Order Count-Min Sketch (HCMS). HCMS [12] is a memory-efficient data structure extended from the Count-Min Sketch (CMS) [31], designed specifically for the approximate storage of entity connection relationships in streaming graphs. As shown in Figure 2, HCMS is structured as a 3-dimensional tensor with dimensions $n_r \times n_b \times n_b$ (layers $\times$ rows $\times$ columns). Each layer, denoted as M_j where j indexes the layer, is associated with two hash functions: h_{r_j} (row hash function) and h_{c_j} (column hash function)³.

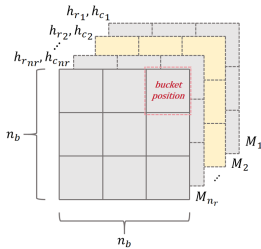


Figure 2: The HCMS Structure

Extended High-Order Count-Min Sketch (EHCMS). To record the timestamp of interactions, we propose EHCMS by extending HCMS, which is specifically a 4-dimensional tensor with dimensions $n_r \times n_b \times n_b \times 2$ (layers $\times$ rows $\times$ columns $\times$ chambers). The key difference is that EHCMS is essentially a variant of HCMS, in which the element of bucket position is replaced by a 2-dimensional vector⁴.

4.2.2 Core Operations.

³In HCMS, the storage position of a single element is referred to as a bucket position.

⁴In EHCMS, the storage position of a single vector is referred to as a bucket position.

Algorithm 2: Core Operations of EHCMS

```

1 Procedure Initialize( $n_r, n_b$ ):
2   for  $j \leftarrow 1$  to  $n_r$  do
3      $h_{r_j} : \mathcal{V} \rightarrow [0, n_b], h_{c_j} : \mathcal{V} \rightarrow [0, n_b]$ 
4      $M_j \leftarrow [0, 0]_{n_b \times n_b}$ 
5 Procedure Update( $e_i = (u_i, v_i, w_i, t_i)$ ):
6   for  $j \leftarrow 1$  to  $n_r$  do
7      $M_j[h_{r_j}(u_i)][h_{c_j}(v_i)][0] \leftarrow M_j[h_{r_j}(u_i)][h_{c_j}(v_i)][0] + w_i$ 
8      $M_j[h_{r_j}(u_i)][h_{c_j}(v_i)][1] \leftarrow t_i$ 

```

Core Operations of HCMS. HCMS supports three core operations, detailed in Algorithm 1:

- (1) **Initialization:** Assign zero values to a predefined HCMS structure and assign random hash functions to each layer;
- (2) **Update:** Upon the arrival of a streaming edge e_i , update the weight count of the corresponding bucket in each layer of HCMS with w_i ;
- (3) **Decay:** Upon a timestamp change, multiply all bucket values in HCMS by a fixed decay factor γ , serving to weaken the impact of old interactions.

Core Operations of EHCMS. EHCMS supports two core operations, detailed in Algorithm 2:

- (1) **Initialization:** Assign $[0, 0]$ (a 2-dimensional vector, where the first dimension is for weight count and the second for timestamp record) to each bucket of a predefined EHCMS structure, and assign random hash functions to each layer;
- (2) **Update:** Upon the arrival of a streaming edge e_i , update the weight count (first dimension) using w_i and the timestamp record (second dimension) using t_i for the corresponding bucket in each layer of EHCMS.

Explanation of Update Operations. Upon the arrival of a streaming edge $e_i = (u_i, v_i, w_i, t_i)$, the update operations for HCMS and EHCMS are detailed as follows:

- (1) HCMS: For the j -th layer M_j of HCMS (see Figure 3 for the update process illustration), the row hash function h_{r_j} maps the source node u_i to row position $h_{r_j}(u_i)$, and the column hash function h_{c_j} maps the destination node v_i to column position $h_{c_j}(v_i)$. The weight count in bucket $M_j[h_{r_j}(u_i)][h_{c_j}(v_i)]$ is then updated by adding the weight w_i of edge e_i (defaulting to 1 if unspecified).
- (2) EHCMS: The key difference from HCMS is updating the 2-dimensional vector in bucket $M_j[h_{r_j}(u_i)][h_{c_j}(v_i)]$: the first dimension $M_j[h_{r_j}(u_i)][h_{c_j}(v_i)][0]$ follows HCMS's weight update (adding w_i of e_i); the second dimension $M_j[h_{r_j}(u_i)][h_{c_j}(v_i)][1]$ is set to the timestamp t_i of e_i .

4.2.3 Count Estimation Guarantees. While HCMS and EHCMS have constant, user-controllable space complexity, determined by n_r and n_b , hash collisions may cause bucket count overestimation. To mitigate this, the final count estimate for an entity pair (u_i, v_i) is defined as the minimum count value across its corresponding buckets in all HCMS/EHCMS layers. Specifically, for any directed interaction from u_i to v_i , let $y(u_i, v_i)$ denote the true count

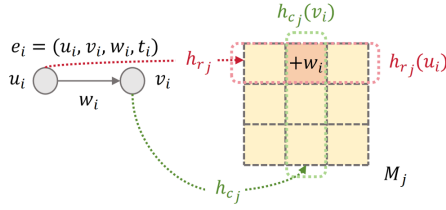


Figure 3: The Update Operation for The j -th Layer of HCMS

of such interactions up to the current timestamp. Let $\hat{y}(u_i, v_i)$ denote the estimate from HCMS/EHCMS: for the HCMS, $\hat{y}(u_i, v_i) = \min_{j \in [1, n_r]} M_j[h_{r_j}(u_i)][h_{c_j}(v_i)]$; and for the EHCMS, $\hat{y}(u_i, v_i) = \min_{j \in [1, n_r]} M_j[h_{r_j}(u_i)][h_{c_j}(v_i)][0]$.

Due to hash collisions, $\hat{y}(u_i, v_i)$ tends to overestimate the true count, i.e., $y(u_i, v_i) \leq \hat{y}(u_i, v_i)$. Let $\mathcal{M} = \sum_{u_i, v_i} y(u_i, v_i)$ denote the total true count of observed directed interactions to date. Lemma 4.2 [12] shows that HCMS and EHCMS provide estimation guarantees analogous to the Count-Min Sketch (CMS). This ensures that even after approximate streaming graph storage, relevant metrics can be effectively estimated, thereby safeguarding the performance of the proposed detection algorithms.

LEMMA 4.2. *For all $j \in [n_r]$, let $h_j(u_i, v_i) = (h_{r_j}(u_i), h_{c_j}(v_i))$, where each of the row hash functions h_{r_j} and column hash functions h_{c_j} is selected uniformly at random from a pairwise-independent family. Here, we allow both cases of $h_r = h_c$ and $h_r \neq h_c$. Fix a confidence parameter $\delta > 0$ and an error bound $\epsilon > 0$, and then set $n_r = \lceil \ln \frac{1}{\delta} \rceil$ and $n_b = \lceil \frac{\epsilon}{\epsilon} \rceil$. With probability at least $1 - \delta$, the following holds for any entity pair (u_i, v_i) : $\hat{y}(u_i, v_i) \leq y(u_i, v_i) + \epsilon \mathcal{M}$.*

4.3 Detection Algorithms

Building on the Interaction Willingness (IW) metric and the implicit streaming graph storage structures (HCMS and EHCMS), this section elaborates on the design rationales and two anomaly detection algorithms: AnoD and AnoC.

4.3.1 AnoD: Dense-type Anomalous Interaction Detection.

Design Rationale. AnoD is designed based on the following core logic: Under a global decay mechanism, where the impact of old interaction behaviors (i.e., the accumulated weight counts of old streaming edges) decays continuously over time (filtering out normal long-term frequent behaviors), a high IW strength between the interacting entities (i.e., the source node u_i and target node v_i of the streaming edge e_i) at a given timestamp signifies a high propensity for directed high-frequency interactions between these entities in the recent period. That is, the current interaction (i.e., the streaming edge e_i) is highly likely to correspond to a dense-type anomalous interaction behavior.

Algorithm Design. AnoD adopts HCMS for the approximate storage of streaming graphs, and its pseudocode is provided in Algorithm 3. The workflow of AnoD is detailed as follows: (1) Initialize an HCMS structure for weight counting, which is consistent with the initialization step in Algorithm 1; (2) For each newly incoming streaming edge $e_i = (u_i, v_i, w_i, t_i)$: (a) If the timestamp t_i of e_i differs from the timestamp of the last processed edge, execute the Decay

operation of HCMS (a global decay mechanism) to weaken the influence of old interactions; (b) Update the corresponding buckets in HCMS using the weight w_i of e_i (following the Update operation logic of HCMS); (c) Estimate the IW strength $\hat{I}(u_i, v_i)$ based on the updated HCMS, and use this estimated value directly as the anomaly score $S(e_i)$ for the edge e_i .

Estimation of IW Strength and Its Theoretical Guarantee. The method for estimating the IW strength $I(u_i, v_i)$, for any directed node pair where u_i acts as the source and v_i as the destination, in the HCMS is formally defined in Equation (2). Here, $\hat{I}(u_i, v_i)$ denotes the estimated IW strength.

$$\hat{I}(u_i, v_i) = \min_{j \in [1, n_r]} \left(\sum_{p \in [0, n_b - 1]} M_j[h_{r_j}(u_i)][p] + \sum_{q \in [0, n_b - 1]} M_j[q][h_{c_j}(v_i)] \right) \quad (2)$$

This is because, in each layer of HCMS, the row hash function (designed for source nodes) or column hash function (designed for destination nodes) each consistently maps the same node to a fixed row or column position, respectively. Therefore, the sum of weight counts in the row corresponding to u_i 's row hash position $h_{r_j}(u_i)$ serves as an estimate of u_i 's out-degree; similarly, the sum of weight counts in the column corresponding to v_i 's column hash position $h_{c_j}(v_i)$ serves as an estimate of v_i 's in-degree. To alleviate the overestimation bias induced by hash collisions, we adopt the minimum value of the estimated IW strength from all HCMS layers as the final estimated value $\hat{I}(u_i, v_i)$.

We provide a theoretical guarantee for HCMS-based IW strength estimation, which is formalized as Theorem 4.3 (with detailed proofs included in Appendix A). This further ensures the detection performance of the AnoD algorithm.

Algorithm 3: AnoD

Input : Streaming edges (E) arriving over time

Output : Anomaly score per edge

```

1 Procedure Anomaly Detection ( $E$ )
2   Initialize HCMS structure  $\mathbf{M}$  for edges count
3   while new edge  $e_i = (u_i, v_i, w_i, t_i) \in E$  do
4     Decay HCMS with timestamp change
5     Update HCMS for new edge  $e_i$  with value  $w_i$ 
6      $S(e_i) \leftarrow$  SAnoD Score Calculate ( $\mathbf{M}, e_i = (u_i, v_i)$ )
7     return  $S(e_i)$ 

8 Procedure SAnoD Calculate ( $\mathbf{M}, e_i = (u_i, v_i)$ )
9   for  $j \leftarrow 1$  to  $n_r$  do
10     $\hat{d}_{out_j}(u_i) \leftarrow 0, \hat{d}_{in_j}(v_i) \leftarrow 0$ 
11    for  $p, q \leftarrow 0$  to  $n_b$  do
12       $\hat{d}_{out_j}(u_i) \leftarrow \hat{d}_{out_j}(u_i) + \mathbf{M}_j[h_{r_j}(u_i)][p]$ 
13       $\hat{d}_{in_j}(v_i) \leftarrow \hat{d}_{in_j}(v_i) + \mathbf{M}_j[q][h_{c_j}(v_i)]$ 
14     $\hat{I}_j(u_i, v_i) \leftarrow \hat{d}_{out_j}(u_i) + \hat{d}_{in_j}(v_i)$ 
15     $\hat{I}(u_i, v_i) \leftarrow \min(\hat{I}_j(u_i, v_i))$ 
16    return  $\hat{I}(u_i, v_i)$ 

```

THEOREM 4.3. *For any confidence parameter $\delta > 0$ and error bound $c > 0$, by setting $n_b = \lceil \frac{eM}{c} \rceil$ and $n_r = \lceil \frac{\ln(1/\delta)}{\ln(e/2)} \rceil$, the following holds: for any directed node pair (u_i, v_i) , the error of the IW estimate satisfies $|\hat{I}(u_i, v_i) - I(u_i, v_i)| \leq c$ with probability at least $1 - \delta$.*

4.3.2 AnoC: Counter-type Anomalous Interaction Detection.

Design Rationale. AnoC is designed based on the following core logic: The anomaly score of an interaction at a given timestamp can be determined by the difference between the actual interaction frequency and the estimated frequency, which measures the deviation level from its long-term behavior, i.e., the possibility of a counter-type interaction. Therefore, in AnoC, we construct a variant of IW, namely Frequency-based Interaction Willingness (FIW), by integrating the newly proposed interaction frequency computation with an element-wise decay mechanism. This FIW can effectively capture the magnitude of the estimated directed interaction frequency between entities at the current moment. Furthermore, the additional timestamp records stored in the Extended High-Order Count-Min Sketch (EHCMS) support the calculation of the actual interaction frequency and the implementation of the element-wise decay mechanism, ensuring the feasibility of the algorithm.

Before proposing AnoC, we first introduce the concepts of interaction frequency and FIW.

Interaction Frequency and FIW. Interaction frequency is calculated by temporally normalizing the weight count of directed interactions from node u_i to v_i by the last occurrence time of such interactions, as formalized in Equation 3. Here, $y(u_i, v_i)$ denotes the weight count of directed interactions from u_i to v_i at the current timestamp, and $t_f(u_i, v_i)$ denotes the timestamp of the last observed interaction from u_i to v_i up to the current timestamp.

$$f(u_i, v_i) = \frac{y(u_i, v_i)}{t_f(u_i, v_i)} \quad (3)$$

Based on the computation of interaction frequency, we further define the Frequency-based Interaction Willingness (FIW) between entities (i.e., nodes) via an element-wise decay mechanism, whose formal definition is presented in Definition 4.4.

Definition 4.4 (Frequency-based Interaction Willingness). For a streaming graph G corresponding to a system state at timestamp t_i , the FIW strength $FI(u_i, v_i)$ between a directed node pair u_i (source) and v_i (destination) is calculated as shown in Equation 4:

$$FI(u_i, v_i) = fd_{out}(u_i) + fd_{in}(v_i) \quad (4)$$

where $fd_{out}(u_i)$ refers to the frequency out-degree of node u_i and $fd_{in}(v_i)$ refers to the frequency in-degree of node v_i ⁵. Their formal expressions are given in Equations 5 and 6, respectively. Here: $N_o(u_i)$ denotes the set of nodes directly pointed to by node u_i ; $N_u(v_i)$ denotes the set of nodes directly pointing to node v_i ; $d_f(u_i, v_i)$ is the element-wise decay factor associated with the interaction frequency $f(u_i, v_i)$, whose calculation is formalized in Equation 7, where b is a tunable offset constant and k is a tunable decay coefficient. Specifically, the element-wise decay factor $d_f(u_i, v_i)$ maps the time difference between the current timestamp t_i and $t_f(u_i, v_i)$ (the timestamp of the last u_i -to- v_i interaction) to

⁵Similar to IW, FIW can also be generalized to a weighted sum form using parameters α and β .

a value within $(0, 1)$, while preserving the monotonic relationship between the time difference and the decay factor.

Interaction frequency quantifies the temporal average activity intensity of directed interactions from u_i to v_i . The element-wise decay mechanism mitigates the impact of long-period inactive interaction behaviors. Thus, the FIW strength, serving as a variant of IW and derived by integrating these two components, quantifies the expected likelihood of the directed interaction between the corresponding nodes at the current timestamp. A higher FIW strength indicates a greater likelihood of interaction.

$$fd_{out}(u_i) = \sum_{v_i \in N_o(u_i)} f(u_i, v_i) \cdot d_f(u_i, v_i) \quad (5)$$

$$fd_{in}(v_i) = \sum_{u_i \in N_u(v_i)} f(u_i, v_i) \cdot d_f(u_i, v_i) \quad (6)$$

$$d_f(u_i, v_i) = e^{-\frac{|t_i - t_f(u_i, v_i)| + b}{k}} \quad (7)$$

Algorithm Design. AnoC employs EHCMS for the approximate storage of streaming graphs, and its pseudocode is provided in Algorithm 4. The workflow of AnoC is detailed as follows: (1) Initialize an EHCMS structure (consistent with the initialization step in Algorithm 2); (2) For each newly incoming streaming edge $e_i = (u_i, v_i, w_i, t_i)$: (a) Execute an update operation to refresh the weight count and timestamp record in the corresponding EHCMS buckets; (b) Estimate the interaction frequency $\hat{f}(u_i, v_i)$ associated with this edge; (c) Compute the FIW strength between the directed entity pair (u_i, v_i) corresponding to this edge via the element-wise decay mechanism. (3) Calculate the difference between the normalized FIW strength and the interaction frequency of the current edge as the anomaly score $S(e_i)$ for the edge e_i . This is because a larger FIW indicates that the interaction frequency corresponding to the current interaction behavior should be higher, while a lower interaction frequency suggests that this interaction behavior does not align with the expectations based on its long-term interaction patterns.

Estimation of FIW Strength. To estimate the FIW strength, we use the timestamp records stored in EHCMS as approximate values for the timestamps of the last occurrence of corresponding directed interactions up to the current timestamp. The specific estimation method is detailed in the "**Procedure FIW Calculate**" part of Algorithm 4. It is important to note that EHCMS provides theoretical guarantees for weight count estimation (as discussed in Section 4.2.3), while its timestamp records may have minor errors due to hash collisions—an inherent trait of hash-based structures. The analysis of timestamp errors in Appendix B shows that this issue can be effectively alleviated by increasing n_r and n_b in EHCMS.

4.4 Complexity Analysis and Edge Deletions

Both AnoD and AnoC exhibit $O(1)$ space complexity and constant time complexity for processing a single streaming edge. Detailed analysis is provided in Appendix C.

Please note that both AnoD and AnoC are compatible with edge deletion operations. We can simply replace the $+w_i$ with $-w_i$ in the update operations of the corresponding structures (HCMS or

Algorithm 4: AnoC

Input : Streaming edges (E) arriving over time
Output : Anomaly score per edge

- 1 **Procedure** *Anomaly Detection* (E)
- 2 Initialize EHCMS structure $\mathbf{M}$ for edges count and timestamps record
- 3 **while** new edge $e_i = (u_i, v_i, w_i, t_i) \in E$ **do**
- 4 Update EHCMS for new edge e_i with value w_i and timestamp t_i
- 5 $S(e_i) \leftarrow \text{SAnoC Calculate}(\mathbf{M}, e_i = (u_i, v_i, t_i))$
 return $S(e_i)$
- 6 **Procedure** *SAnoC Calculate* ($\mathbf{M}, e_i = (u_i, v_i, t_i)$)
- 7 **for** $j \leftarrow 1$ **to** n_r **do**
- 8 $\hat{f}_j(u_i, v_i) \leftarrow \frac{M_j[h_{r_j}(u_i)][h_{c_j}(v_i)][0]}{M_j[h_{r_j}(u_i)][h_{c_j}(v_i)][1]}$
- 9 $\hat{f}(u_i, v_i) \leftarrow \min(\hat{f}_j(u_i, v_i))$
- 10 $\widehat{\text{FI}}(u_i, v_i) \leftarrow \text{FIW Calculate}(e_i = (u_i, v_i, t_i))$
- 11 $n\widehat{\text{FI}}(u_i, v_i) \leftarrow \frac{\widehat{\text{FI}}(u_i, v_i)}{2 \cdot n_b}$
- 12 **return** $n\widehat{\text{FI}}(u_i, v_i) - \hat{f}(u_i, v_i)$
- 13 **Procedure** *FIW Calculate* ($e_i = (u_i, v_i, t_i)$)
- 14 **for** $j \leftarrow 1$ **to** n_r **do**
- 15 $\widehat{f}_{out_j}(u_i) \leftarrow 0, \widehat{f}_{in_j}(v_i) \leftarrow 0$;
- 16 **for** $p \leftarrow 0$ **to** n_b **do**
- 17 $\widehat{f}_{out_j}(u_i) \leftarrow \widehat{f}_{out_j}(u_i) + \left(\frac{M_j[h_{r_j}(u_i)][p][0]}{M_j[h_{r_j}(u_i)][p][1]} \right) \cdot e^{-\frac{(t_i - M_j[h_{r_j}(u_i)][p][1]) + b}{k}}$
- 18 **for** $q \leftarrow 0$ **to** n_b **do**
- 19 $\widehat{f}_{in_j}(v_i) \leftarrow \widehat{f}_{in_j}(v_i) + \left(\frac{M_j[q][h_{c_j}(v_i)][0]}{M_j[q][h_{c_j}(v_i)][1]} \right) \cdot e^{-\frac{(t_i - M_j[q][h_{c_j}(v_i)][1]) + b}{k}}$
- 20 $\widehat{\text{FI}}_j(u_i, v_i) \leftarrow \widehat{f}_{out_j}(u_i) + \widehat{f}_{in_j}(v_i)$
- 21 $\widehat{\text{FI}}(u_i, v_i) \leftarrow \min(\widehat{\text{FI}}_j(u_i, v_i))$
- 22 **return** $\widehat{\text{FI}}(u_i, v_i)$

EHCMS). The deletion operation for a single edge incurs a computational complexity of $O(1)$. In certain scenarios (e.g., unfollowing in social networks), assisting in edge deletion operations can improve the accuracy of the subsequent calculation of anomaly scores for streaming edges.

5 Experiments

In this section, we evaluate the performance of the proposed methods against the state-of-the-art rule-based methods and seek to address the following research questions (**RQs**):

- **RQ1:** How is the accuracy and efficiency of AnoD and AnoC in detecting their target anomalous interaction types?
- **RQ2:** Are AnoD and AnoC sufficiently scalable and robust on different scales of graphs?
- **RQ3:** How do different hyperparameter settings affect the detection performance of AnoD and AnoC?

5.1 Experimental Setup

Datasets. We evaluate on the public real-world datasets with anomaly labels. Table 5 of Appendix E presents the statistical characteristics of the datasets in which 4 datasets may contain dense-type anomalies and 2 datasets may contain counter-type anomalies. **Dense-type Datasets:** Four classic datasets (DARPA [32], ISCX-IDS2012 [33], CIC-IDS2018 [34], and CIC-DDoS2019 [35]) are used to evaluate the detection performance of dense-type anomalies. In these datasets, network attack behaviors (e.g., DDoS, port scanning) initiated by malicious IPs against target IPs are labeled as anomalous. **Counter-type Datasets:** Two existing datasets (Wikipedia and Reddit [36]) are used to evaluate the detection performance of counter-type anomalies. Wikipedia records users' Wikipedia page-editing behaviors: an editing behavior is labeled anomalous if the user is banned after performing it. Reddit captures users' posting behaviors: a post is labeled anomalous if the user is banned after its submission.

Competitors. To comprehensively assess the performance of the proposed metric and algorithms, 7 evaluation rule-based methods (as introduced in Section 2) are compared.

Detailed hyperparameter settings for our work and these competitors are given in Appendix D.

Experimental Evaluation Metrics. Consistent with prior works, we employ AUC, i.e., the Area Under the Receiver Operating Characteristic (ROC) Curve, to measure detection accuracy, and runtime to assess efficiency and scalability. In anomaly detection, threshold selection depends on the specific requirements of real-world tasks. Notably, AUC calculation is threshold-independent, enabling robust evaluation of overall detection performance.

Experimental Configuration. AnoD and AnoC were implemented in Python 3.12. All experiments were conducted on a computer configured with a 12th Gen Intel(R) Core(TM) i7-12700 processor (2.10 GHz) and 32.0 GB of RAM. All experiments (including competitors' runs) were repeated five times, with mean values reported⁶.

5.2 Performance Evaluation (RQ1 & RQ2)

Accuracy and Running Time (RQ1). As shown in Table 1, AnoD achieves higher accuracy than all the compared methods across dense-type datasets⁷ while maintaining a relatively low runtime (total runtime on processing all the edges in each dataset)⁸. In contrast, AnoC delivers strong detection performance on counter-type datasets, with a slightly longer runtime. Notably, other than AnoD and AnoEdge-G, no method can effectively detect anomalous interactions across all dense-type datasets (with AUC values consistently above 0.9); Similarly, other than AnoC, no method can effectively detect anomalous interactions across all counter-type datasets (with AUC values consistently above 0.6). This highlights the effectiveness of AnoD and AnoC in detecting their respective types of anomalous interactions. Furthermore, we find that AnoD and AnoC do not

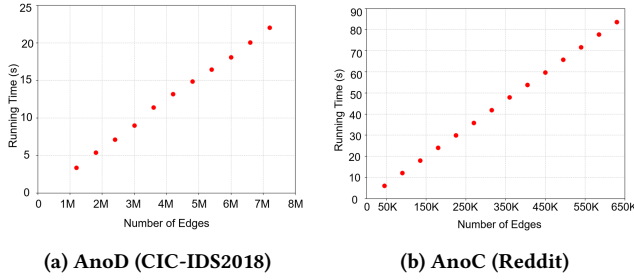
⁶The standard deviation of the results across the five repetitions was negligible compared to the mean values, and thus is not separately presented.

⁷On datasets with over 92% normal edges (ISCX-IDS2012 and CIC-IDS2018), if we take the top 5% anomaly score as the threshold, the false positive rate is below 1.5% and the accuracy is above 95.5%.

⁸PENminer failed to process the CIC-DDoS2019 dataset within 24 hours, so its AUC is not reported.

Table 1: AUC and Total Running Time on All the Datasets

Dataset	DenseAlert	SedanSpot	Midas-N	MStream	PENminer	F-FADE	AnoEdge-G	AnoD	AnoC
DARPA	0.856 31.69s	0.638 138.87s	0.895 4.66s	0.954 1.06s	0.872 5.3h	0.918 398.56s	0.971 27.32s	0.975 15.31s	0.148 817.02s
ISCX-IDS2012	0.587 62.22s	0.582 17.62s	0.448 1.41s	0.645 0.26s	0.530 1.5h	0.530 159.05s	0.954 8.21s	0.957 21.51s	0.306 204.39s
CIC-IDS2018	0.756 281.98s	0.376 175.56s	0.637 7.37s	0.758 1.88s	0.821 10.1h	0.606 517.66s	0.963 61.06s	0.984 22.33s	0.156 1512.63s
CIC-DDoS2019	0.287 223.46s	0.567 670.86s	0.724 23.83s	0.760 4.86s	- >24h	0.668 34.60s	0.998 114.25s	0.999 43.73s	0.001 3284.83s
Reddit	0.683 178.53s	0.566 11.89s	0.569 1.62s	0.651 0.17s	0.451 2.8h	0.498 258.31s	0.468 8.27s	0.493 63.44s	0.616 94.91s
Wikipedia	0.488 9.14s	0.727 1.61s	0.646 0.55s	0.483 0.04s	0.477 1.2h	0.500 213.37s	0.441 2.29s	0.451 18.82s	0.754 21.40s

**Figure 4: Linear Scalability with the Number of Edges**

achieve ideal accuracy on the other type of dataset (i.e., AnoD on counter-type datasets and AnoC on dense-type datasets), which suggests that there are considerable differences between the two types of anomalies. The aforementioned experimental results verify the two types of anomalies are prevalent in the evaluated datasets, as well as the effectiveness of the corresponding detection algorithms designed for them.

Scalability (RQ2). Figures 4a and 4b plot the running time of AnoD (on the CIC-IDS2018 dataset) and AnoC (on the Reddit dataset) against the number of incoming streaming edges, respectively. These figures confirm the scalability of AnoD and AnoC, demonstrating that running time scales linearly with the number of streaming edges, a trend that is consistent across all evaluated datasets.

Robustness (RQ2). Table 6 of Appendix E reports the AUC values of AnoD (on the DARPA and ISCX-IDS2012 datasets) and AnoC (on the Reddit and Wikipedia datasets) across different data ratios (i.e., the proportions of the complete dataset used for evaluation). It can be observed that our algorithms are sufficiently robust (i.e., the AUC values fluctuate less on different proportions of data).

5.3 Hyperparameter Analysis (RQ3)

Varying γ . Table 7 of Appendix E reports the AUC values of AnoD on dense-type datasets across different values of the global decay factor γ . Results indicate that the AUC of AnoD decreases significantly when $\gamma = 1$, which confirms the necessity of the global decay mechanism. Furthermore, when $\gamma = 0.2$, the AUC also exhibits a noticeable decrease, which is attributed to the fact

that an excessively small γ leads to the excessive loss of historical interaction information.

Varying n_r and n_b . Experimental results in Table 8 of Appendix E present the AUC (Running Time) of AnoD (on the DARPA dataset) and AnoC (on the Wikipedia dataset) under varying values of n_r and n_b , respectively. It reveals that algorithm performance improves with increasing n_r and n_b , indicating that larger n_r and n_b can mitigate the impact of hash collisions to some extent.

Supplementary Analysis. The impact of AnoC’s offset constant b and decay coefficient k , as well as AnoD’s weight parameters α and β , is detailed in Appendix E. Overall, both AnoD and AnoC exhibit strong parameter robustness under different hyperparameter settings with only minor AUC fluctuations. As a core advantage of rule-based methods, interpretability verification for our proposed approach is additionally detailed in Appendix E.

6 Conclusion

This paper proposes a simple, effective, and efficient evaluation metric, namely Interaction Willingness (IW), and two real-time algorithms (AnoD and AnoC) based on this metric. The algorithms can specifically detect two common types of anomalous interactions (dense-type and counter-type), which cover a variety of typical anomalous interaction behaviors in real networks. The time and space complexities of our algorithms are both constant, which supports real-time processing of streaming graphs. Although this lightweight design enables seamless integration into real-world systems, real-time data ingestion via a dedicated adapter is required to preserve temporal consistency during anomaly detection. In future work, we plan to integrate the learning-based and rule-based methods to study the detection of more general types of anomalies.

Acknowledgments

This work is partially supported by the National Natural Science Foundation of China (62572140, U2436208, U25B2047, 62272469), the Guangdong Basic and Applied Basic Research Foundation (2024A1515011501), the Guangdong S&T Programme under Grant 2024B0101010002, and the Project of Guangdong Key Laboratory of Industrial Control System Security (2024B1212020010). We thank the anonymous reviewers for their insightful suggestions.

References

- [1] Kaiyu Chen, Dong Wen, Wenjie Zhang, Ying Zhang, Xiaoyang Wang, and Xuemin Lin. 2024. Querying Structural Diversity in Streaming Graphs. *Proc. VLDB Endow.* 17, 5 (2024), 1034–1046.
- [2] Xiaolin Han, Xiurui Hu, Chenhao Ma, and Xuequn Shang. 2025. ReadGraph: Relational Evolution Enhanced Anomaly Detection in Dynamic Heterogeneous Graph. In *Companion Proceedings of the WWW 2025, Sydney, NSW, Australia, April 28 - May 2, 2025*. 1009–1013.
- [3] Huan Wang, Junyang Chen, Yirui Wu, Victor C. M. Leung, and Di Wang. 2025. EPM: Evolutionary Perception Method for Anomaly Detection in Noisy Dynamic Graphs. *IEEE Trans. Knowl. Data Eng.* 37, 7 (2025), 4035–4048.
- [4] Hezhe Qiao, Hanghang Tong, Bo An, Irwin King, Charu Aggarwal, and Guansong Pang. 2025. Deep Graph Anomaly Detection: A Survey and New Perspectives. *IEEE Trans. Knowl. Data Eng.* 37, 9 (2025), 5106–5126.
- [5] Mati Ur Rehman, Hadi Ahmadi, and Wajih Ul Hassan. 2024. Flash: A Comprehensive Approach to Intrusion Detection via Provenance Graph Representation Learning. In *IEEE SP 2024, San Francisco, CA, USA, May 19-23, 2024*. 3552–3570.
- [6] Xiao Yang, Xuejiao Zhao, and Zhiqi Shen. 2025. A Generalizable Anomaly Detection Method in Dynamic Graphs. In *AAAI 2025, Philadelphia, PA, USA, February 25 - March 4, 2025*. 22001–22009.
- [7] Jianke Yu, Hanchen Wang, Xiaoyang Wang, Zhao Li, Lu Qin, Wenjie Zhang, Jian Liao, Ying Zhang, and Bailin Yang. 2025. Temporal Insights for Group-Based Fraud Detection on e-Commerce Platforms. *IEEE Trans. Knowl. Data Eng.* 37, 2 (2025), 951–965.
- [8] Abdinasir Hirsir Abdi, Mohammed A. Alhartomi, Lukman Audah, Adeb Salh, Nan Mad Sahar, Salman Ahmed, Godwin O. Ansa, and Abdullahi Farah. 2025. Comprehensive Analysis of DDoS Anomaly Detection in Software-Defined Networks. *IEEE Access* 13 (2025), 23013–23071.
- [9] Mohammad Ziari and Nasrollah Moghaddam Charkari. 2026. Rumor detection and propagation on social networks: A survey. *Expert Syst. Appl.* 295 (2026), 128798.
- [10] Cheng Wang and Hangyu Zhu. 2022. Wrongdoing Monitor: A Graph-Based Behavioral Anomaly Detection in Cyber Security. *IEEE Trans. Inf. Forensics Secur.* 17 (2022), 2703–2718.
- [11] Imen Souiden, Mohamed Nazih Omri, and Zaki Brahmi. 2022. A survey of outlier detection in high dimensional data streams. *Comput. Sci. Rev.* 44 (2022), 100463.
- [12] Siddharth Bhatia, Mohit Wadhwa, Kenji Kawaguchi, Neil Shah, Philip S. Yu, and Bryan Hooi. 2023. Sketch-Based Anomaly Detection in Streaming Graphs. In *KDD 2023, Long Beach, CA, USA, August 6-10, 2023*. 93–104.
- [13] Wenchao Yu, Wei Cheng, Charu C. Aggarwal, Kai Zhang, Haifeng Chen, and Wei Wang. 2018. NetWalk: A Flexible Deep Embedding Approach for Anomaly Detection in Dynamic Networks. In *KDD 2018, London, UK, August 19-23, 2018*. 2672–2681.
- [14] Li Zheng, Zhenpeng Li, Jian Li, Zhao Li, and Jun Gao. 2019. AddGraph: Anomaly Detection in Dynamic Graph Using Attention-based Temporal GCN. In *IJCAI 2019, Macao, China, August 10-16, 2019*. 4419–4425.
- [15] Yixin Liu, Shirui Pan, Yu Guang Wang, Fei Xiong, Liang Wang, Qingfeng Chen, and Vincent Cheng-Siong Lee. 2023. Anomaly Detection in Dynamic Graphs via Transformer. *IEEE Trans. Knowl. Data Eng.* 35, 12 (2023), 12081–12094.
- [16] Sheng Tian, Jihai Dong, Jintang Li, Wenlong Zhao, Xiaolong Xu, Baokun Wang, Bowen Song, Changhua Meng, Tianyi Zhang, and Liang Chen. 2023. SAD: Semi-Supervised Anomaly Detection on Dynamic Graphs. In *IJCAI 2023, Macao, SAR, China, August 19-25, 2023*. 2306–2314.
- [17] Lei Cai, Zhengzhang Chen, Chen Luo, Jiaping Gui, Jingchao Ni, Ding Li, and Haifeng Chen. 2021. Structural Temporal Graph Neural Networks for Anomaly Detection in Dynamic Graphs. In *CIKM 2021, Virtual Event, Queensland, Australia, November 1-5, 2021*. 3747–3756.
- [18] Jongha Lee, Sunwoo Kim, and Kijung Shin. 2024. SLADE: Detecting Dynamic Anomalies in Edge Streams without Labels via Self-Supervised Learning. In *KDD 2024, Barcelona, Spain, August 25-29, 2024*. 1506–1517.
- [19] Ocheme Anthony Ekle and William Eberle. 2024. Anomaly Detection in Dynamic Graphs: A Comprehensive Survey. *ACM Trans. Knowl. Discov. Data* 18, 8 (2024), 192:1–192:44.
- [20] Junquan Gu, Hang Yu, and Xiangfeng Luo. 2025. A Masked AutoEncoder with Strong-Weak Mutual Information for Anomaly Detection in Dynamic Incomplete Graphs. In *Companion Proceedings of the WWW 2025, Sydney, NSW, Australia, April 28 - May 2, 2025*. 986–990.
- [21] Xi He, Kai Huang, Qingqing Ye, and Haibo Hu. 2025. Data Poisoning Attacks to Local Differential Privacy Protocols for Graphs. In *ICDE 2025, Hong Kong, May 19-23, 2025*. 987–1000.
- [22] Yihong Huang, Yuang Zhang, Liping Wang, Fan Zhang, and Xuemin Lin. 2024. EntropyStop: Unsupervised Deep Outlier Detection with Loss Entropy. In *KDD 2024, Barcelona, Spain, August 25-29, 2024*. 1143–1154.
- [23] Stephen Ranshous, Steve Harenberg, Kshitij Sharma, and Nagiza F. Samatova. 2016. A Scalable Approach for Outlier Detection in Edge Streams Using Sketch-based Approximations. In *SDM 2016, Miami, Florida, USA, May 5-7, 2016*. 189–197.
- [24] Kumar Sricharan and Kamalika Das. 2014. Localizing anomalous changes in time-evolving graphs. In *SIGMOD 2014, Snowbird, UT, USA, June 22-27, 2014*. 1347–1358.
- [25] Dhivya Eswaran and Christos Faloutsos. 2018. SedanSpot: Detecting Anomalies in Edge Streams. In *ICDM 2018, Singapore, November 17-20, 2018*. 953–958.
- [26] Siddharth Bhatia, Bryan Hooi, Minji Yoon, Kijung Shin, and Christos Faloutsos. 2020. Midas: Microcluster-Based Detector of Anomalies in Edge Streams. In *AAAI 2020, New York, NY, USA, February 7-12, 2020*. 3242–3249.
- [27] Yen-Yu Chang, Pan Li, Rok Susic, M. H. Afifi, Marco Schweighauser, and Jure Leskovec. 2021. F-FADE: Frequency Factorization for Anomaly Detection in Edge Streams. In *WSDM 2021, Virtual Event, Israel, March 8-12, 2021*. 589–597.
- [28] Siddharth Bhatia, Arjit Jain, Pan Li, Ritesh Kumar, and Bryan Hooi. 2021. MStream: Fast Anomaly Detection in Multi-Aspect Streams. In *WWW 2021, Virtual Event, Ljubljana, Slovenia, April 19-23, 2021*. 3371–3382.
- [29] Kijung Shin, Bryan Hooi, Jisu Kim, and Christos Faloutsos. 2017. DenseAlert: Incremental Dense-Subtensor Detection in Tensor Streams. In *KDD 2017, Halifax, NS, Canada, August 13-17, 2017*. 1057–1066.
- [30] Caleb Belth, Xinyi Zheng, and Danai Koutra. 2020. Mining Persistent Activity in Continually Evolving Networks. In *KDD 2020, Virtual Event, CA, USA, August 23-27, 2020*. 934–944.
- [31] Graham Cormode and S. Muthukrishnan. 2005. An improved data stream summary: the count-min sketch and its applications. *J. Algorithms* 55, 1 (2005), 58–75.
- [32] Richard Lippmann, Robert K. Cunningham, David J. Fried, Isaac Graf, Kris R. Kendall, Seth E. Webster, and Marc A. Zissman. 1999. Results of the DARPA 1998 Offline Intrusion Detection Evaluation. In *RAID 1999 Second International Workshop, West Lafayette, Indiana, USA, September 7-9, 1999*.
- [33] Ali Shiravi, Hadi Shiravi, Mahbod Tavallae, and Ali A. Ghorbani. 2012. Toward developing a systematic approach to generate benchmark datasets for intrusion detection. *Comput. Secur.* 31, 3 (2012), 357–374.
- [34] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. 2018. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. In *ICISSP 2018, Funchal, Madeira - Portugal, January 22-24, 2018*. 108–116.
- [35] Iman Sharafaldin, Arash Habibi Lashkari, Saqib Hakak, and Ali A. Ghorbani. 2019. Developing Realistic Distributed Denial of Service (DDoS) Attack Dataset and Taxonomy. In *ICCST 2019, Chennai, India, October 1-3, 2019*. 1–8.
- [36] Srikanth Kumar, Xikun Zhang, and Jure Leskovec. 2019. Predicting Dynamic Embedding Trajectory in Temporal Interaction Networks. In *KDD 2019, Anchorage, AK, USA, August 4-8, 2019*. 1269–1278.

A Proof of Theorem 4.3

PROOF. For generality, we provide the proof for the weighted form.

Error Analysis of Single-Layer Estimation.

Consider the j -th layer of HCMS. Due to hash collisions, the count estimate may overestimate the true value, i.e., $\hat{y}(u_i, v_i) \geq y(u_i, v_i)$.

Decompose the out-degree estimate:

$$\begin{aligned} \hat{d}_{out_j}(u_i) &= \sum_p M_j[h_{r_j}(u_i)] [p] \\ &= d_{out}(u_i) + \sum_{u' \neq u_i} \sum_v y(u', v) \cdot \mathbb{I}[h_{r_j}(u') = h_{r_j}(u_i)], \end{aligned}$$

where $\mathbb{I}[\cdot]$ is the indicator function (equal to 1 if the condition holds, and 0 otherwise).

Similarly, decompose the in-degree estimate:

$$\begin{aligned} \hat{d}_{in_j}(v_i) &= \sum_q M_j[q] [h_{c_j}(v_i)] \\ &= d_{in}(v_i) + \sum_{v' \neq v_i} \sum_u y(u, v') \cdot \mathbb{I}[h_{c_j}(v') = h_{c_j}(v_i)], \end{aligned}$$

Define error terms:

$$\begin{aligned} E_{out_j}(u_i) &= \hat{d}_{out_j}(u_i) - d_{out}(u_i), \\ E_{in_j}(v_i) &= \hat{d}_{in_j}(v_i) - d_{in}(v_i). \end{aligned}$$

Since the hash functions are pairwise independent, we have:

$$\begin{aligned}\mathbb{E}[\mathbb{I}[h_{r_j}(u') = h_{r_j}(u_i)]] &= \frac{1}{n_b}, \\ \mathbb{E}[\mathbb{I}[h_{c_j}(v') = h_{c_j}(v_i)]] &= \frac{1}{n_b},\end{aligned}$$

thus:

$$\begin{aligned}\mathbb{E}[E_{out_j}(u_i)] &\leq \sum_{u' \neq u_i} \sum_v y(u', v) \cdot \frac{1}{n_b} \leq \frac{M}{n_b}, \\ \mathbb{E}[E_{in_j}(v_i)] &\leq \sum_{v' \neq v_i} \sum_u y(u, v') \cdot \frac{1}{n_b} \leq \frac{M}{n_b}.\end{aligned}$$

By Markov's inequality, for any $\varepsilon > 0$:

$$\Pr[E_{out_j}(u_i) \geq \varepsilon] \leq \frac{\mathbb{E}[E_{out_j}(u_i)]}{\varepsilon} \leq \frac{M}{n_b \cdot \varepsilon},$$

set $\varepsilon = \frac{e \cdot M}{n_b}$, substituting this into the inequality gives:

$$\Pr\left[E_{out_j}(u_i) \geq \frac{e \cdot M}{n_b}\right] \leq \frac{1}{e},$$

similarly:

$$\Pr\left[E_{in_j}(v_i) \geq \frac{e \cdot M}{n_b}\right] \leq \frac{1}{e}.$$

Thus, with probability at least $1 - \frac{2}{e}$, the errors in the j -th layer satisfy:

$$E_{out_j}(u_i) \leq \frac{e \cdot M}{n_b}, \quad E_{in_j}(v_i) \leq \frac{e \cdot M}{n_b}$$

Error Control for Multi-Layer Structure.

Define event A_j : the j -th layer satisfies $E_{out_j}(u_i) \leq \frac{e \cdot M}{n_b}$ and $E_{in_j}(v_i) \leq \frac{e \cdot M}{n_b}$.

The probability that all layers fail to satisfy A_j is:

$$\Pr[\forall j \in [n_r], \neg A_j] \leq \left(\frac{2}{e}\right)^{n_r},$$

set this probability to be less than δ , i.e., $\left(\frac{2}{e}\right)^{n_r} \leq \delta$, derive:

$$n_r \geq \left\lceil \frac{\ln(1/\delta)}{\ln(e/2)} \right\rceil.$$

Set

$$n_r = \left\lceil \frac{\ln(1/\delta)}{\ln(e/2)} \right\rceil,$$

the probability that there exists at least one layer j^* satisfying A_{j^*} is at least $1 - \delta$.

In this layer j^* :

$$\begin{aligned}\hat{d}_{out_{j^*}}(u_i) &\leq d_{out}(u_i) + \frac{e \cdot M}{n_b}, \\ \hat{d}_{in_{j^*}}(v_i) &\leq d_{in}(v_i) + \frac{e \cdot M}{n_b}.\end{aligned}$$

Thus:

$$\begin{aligned}\hat{I}_{j^*}(u_i, v_i) &= \alpha \cdot \hat{d}_{out_{j^*}}(u_i) + \beta \cdot \hat{d}_{in_{j^*}}(v_i) \\ &\leq I(u_i, v_i) + (\alpha + \beta) \cdot \frac{e \cdot M}{n_b}.\end{aligned}$$

Since $\hat{I}(u_i, v_i) = \min_j(\hat{I}_j(u_i, v_i)) \leq \hat{I}_{j^*}(u_i, v_i)$, we have:

$$\hat{I}(u_i, v_i) \leq I(u_i, v_i) + (\alpha + \beta) \cdot \frac{e \cdot M}{n_b}$$

On the other hand, the estimate never underestimates the true value (overestimation is caused by hash collisions), so:

$$\hat{I}(u_i, v_i) \geq I(u_i, v_i).$$

Combining these two inequalities gives:

$$|\hat{I}(u_i, v_i) - I(u_i, v_i)| \leq (\alpha + \beta) \cdot \frac{e \cdot M}{n_b}.$$

Control of Error Constant.

Let:

$$c = (\alpha + \beta) \cdot \frac{e \cdot M}{n_b}$$

then:

$$|\hat{I}(u_i, v_i) - I(u_i, v_i)| \leq c$$

By setting:

$$n_b = \left\lceil \frac{e \cdot M}{c} \right\rceil$$

the error bound is controlled to c .

Meanwhile, setting:

$$n_r = \left\lceil \frac{\ln(1/\delta)}{\ln(e/2)} \right\rceil$$

ensures the above conclusion holds with probability at least $1 - \delta$.

Conclusion.

Thus, for any $\delta > 0$ and constant $c > 0$, if we set:

$$n_b = \left\lceil \frac{e \cdot M}{c} \right\rceil, \quad n_r = \left\lceil \frac{\ln(1/\delta)}{\ln(e/2)} \right\rceil$$

then with probability at least $1 - \delta$, for any node pair (u_i, v_i) , the following holds:

$$|\hat{I}(u_i, v_i) - I(u_i, v_i)| \leq c$$

□

B The Analysis of Timestamp Errors in EHCMS

Increasing n_b Reduces Timestamp Error. Let K be the total number of distinct entity pairs in the streaming graph ($K \leq M$, where M is the total number of directed interactions). Hash functions h_{r_j} and h_{c_j} are pairwise independent: For any distinct nodes $u_a \neq u_b$, $\Pr[h_{r_j}(u_a) = h_{r_j}(u_b)] = 1/n_b$; the same holds for h_{c_j} . Define timestamp error rate ϵ_{ts} as the probability that at least one entity pair's timestamp is overwritten due to collisions.

For a single entity pair (u, v) , the probability that another entity pair $(u', v') \neq (u, v)$ maps to the same bucket in layer j is:

$$\begin{aligned}\Pr[h_{r_j}(u') = h_{r_j}(u) \wedge h_{c_j}(v') = h_{c_j}(v)] \\ = \Pr[h_{r_j}(u') = h_{r_j}(u)] \cdot \Pr[h_{c_j}(v') = h_{c_j}(v)] = \frac{1}{n_b^2}\end{aligned}$$

By the union bound (consistent with the Lemma 1 proof), the probability that (u, v) collides with any of the $K - 1$ other entity pairs in one layer is:

$$\Pr[\text{Single pair collision in one layer}] \leq (K - 1) \cdot \frac{1}{n_b^2}$$

it shows that the single-layer collision probability is inversely proportional to n_b^2 . As n_b increases: The number of buckets per layer (n_b^2) grows quadratically, expanding the hash mapping space. The upper bound of the single-layer collision probability decreases

monotonically (e.g., if $K = 10^4$, $n_b = 32$ gives an upper bound of 9.7%, while $n_b = 128$ reduces it to 0.6%).

Fewer collisions in a single layer directly reduce the likelihood of timestamp overwrites, thus lowering ϵ_{ts} .

Increasing n_r Reduces Timestamp Error. The hash functions across EHCMS layers are independent. For an entity pair (u, v) , the probability that it collides in all n_r layers is the product of collision probabilities in each layer:

$$\Pr[\text{Collision in all layers}] = \prod_{j=1}^{n_r} \Pr[\text{Collision in layer } j] \leq \left(\frac{K-1}{n_b^2} \right)^{n_r} \quad (2)$$

EHCMS uses a "minimum estimate" strategy for interaction frequency calculation, which extends to timestamp retrieval: An entity pair's timestamp is accurate if it is stored correctly in at least one layer. The probability of obtaining an accurate timestamp is:

$$\Pr[\text{Accurate timestamp}] = 1 - \Pr[\text{Collision in all layers}] \geq 1 - \left(\frac{K-1}{n_b^2} \right)^{n_r}$$

As n_r increases, the term $\left(\frac{K-1}{n_b^2} \right)^{n_r}$ decays exponentially: If $\frac{K-1}{n_b^2} = 0.1$, $n_r = 1$ gives $\Pr[\text{Accurate timestamp}] \geq 90\%$; $n_r = 3$ raises it to $\geq 99.9\%$; $n_r = 5$ to $\geq 99.999\%$.

This exponential decay ensures that increasing n_r drastically reduces the chance of total timestamp loss, lowering ϵ_{ts} .

Experimental Validation. Table 8 validates the theoretical conclusions by measuring AnoC's AUC (a proxy for timestamp accuracy, as AnoC relies on EHCMS timestamps) on the Wikipedia dataset. We can observe that: (1) Fixed n_r , increasing n_b from 32 to 128 raises AUC by 0.202 (for $n_r = 1$), confirming reduced timestamp error improves detection performance; (2) Fixed n_b , increasing n_r from 1 to 3 raises AUC by 0.084 (for $n_b = 32$), validating multi-layer redundancy mitigates errors.

Conclusion. (1) Increasing n_b reduces single-layer hash collisions by expanding the bucket space, lowering timestamp overwrite chances. (2) Increasing n_r exponentially reduces the probability of collisions across all layers, ensuring accurate timestamp retrieval in at least one layer. (3) Both adjustments synergistically mitigate EHCMS timestamp recording errors, as validated by AnoC's improved AUC on real-world data.

C Complexity Analysis

Space Complexity. **AnoD:** The HCMS structure is a 3-dimensional tensor of size $n_r \times n_b \times n_b$, where n_r (number of layers) and n_b (number of rows/columns per layer) are predefined constants (independent of the size of the streaming graph). Thus, the space complexity of the AnoD algorithm is $O(n_r \cdot n_b^2)$, which is $O(1)$. On the DARPA dataset, the core memory usage is 0.01MB, demonstrating strong competitiveness against existing methods, such as DenseAlert (123.40MB), F-FADE (62.66MB), and SedanSpot (19.23MB).

AnoC: The Extended High-Order Count-Min Sketch (EHCMS) structure is a 4-dimensional tensor of size $n_r \times n_b \times n_b \times 2$, where

n_r and n_b are predefined constants (independent of the size of the streaming graph). Thus, the space complexity of the AnoC algorithm is $O(n_r \cdot n_b^2)$, which is $O(1)$. In practice, the memory usage remains efficient, consuming only 2.75MB on the DARPA dataset.

Time Complexity. **AnoD:** Initialization of HCMS requires setting all buckets to 0, which involves $O(n_r \cdot n_b^2)$ operations. When the timestamp changes, a decay operation (multiplying all bucket values by the decay factor γ) is performed, involving $O(n_r \cdot n_b^2)$ operations. For each newly arriving streaming edge, the corresponding buckets in all layers are updated: each layer requires 2 hash computations and 1 addition operation, resulting in a total of $O(n_r)$ operations. For the scoring process (calculating $\hat{I}(u_i, v_i)$): each layer requires computing the sum of row values and column values (each involving $O(n_b)$ operations) and then taking the minimum value across all layers, resulting in a total of $O(n_b \cdot n_r)$ operations. Thus, the time complexity of AnoD is $O(n_r \cdot n_b^2 + n_r + n_b \cdot n_r)$. Since n_r and n_b are predefined constants, the time complexity of AnoD is constant, i.e., $O(1)$.

AnoC: Initialization of EHCMS requires setting all 2-dimensional vectors to $[0, 0]$, which involves $O(n_r \cdot n_b^2)$ operations. For each newly arriving streaming edge, the corresponding 2-dimensional vectors in all layers are updated: each layer requires 2 hash computations, 1 addition operation, and 1 assignment operation, resulting in a total of $O(n_r)$ operations. For the scoring process: 1) Calculating the interaction frequency of the edge requires n_r division operations and 1 operation to take the minimum value, resulting in $O(n_r)$ operations. 2) Calculating the FIW: each layer requires traversing n_b positions in rows and columns, computing frequency values and decay factors, performing weighted summation, and then taking the minimum value across all layers, resulting in $O(n_b \cdot n_r)$ operations. 3) Calculating the final anomaly score involves $O(1)$ operations. Thus, the time complexity of AnoC is $O(n_r \cdot n_b^2 + n_r + n_b \cdot n_r + 1)$. Since n_r and n_b are predefined constants, the time complexity of AnoC is constant, i.e., $O(1)$.

To further validate the efficiency advantages of the proposed algorithms (AnoD and AnoC) in practical streaming scenarios, we implement and evaluate three representative learning-based methods (AddGraph, TADDY, StrGNN) on the ISCX-IDS2012 dataset for comparative analysis. Experimental results show that our algorithms achieve significant improvements in both memory efficiency and runtime: the memory usage for processing streaming data is more than 50× lower, and the runtime is more than 6× faster compared to these learning-based counterparts. Beyond these efficiency gains, we emphasize that our methods and the learning-based approaches are scenario-complementary; ours are designed for resource-constrained, interpretability-driven scenarios, while the latter are preferred for resource-abundant settings where capturing complex anomaly patterns is the priority.

D Hyperparameter Settings

For our methods, table 2 presents the hyperparameter settings for AnoD and AnoC. For baseline methods, we use the open-source implementations of AnoEdge-G [12], SedanSpot [25], MIDAS-N [26], F-FADE [27], MStream [28], DenseAlert [29], and PENminer [30]

Table 2: Hyperparameter Settings

Algorithm	Hyperparameter Settings
AnoD	$n_r = 2, n_b = 32; \gamma = 0.9$
AnoC	$n_r = 2, n_b = 128; b = 1, k = 1$

provided by the authors, with hyperparameters set as recommended in the original papers:

- **AnoEdge-G**: $n_r = 2, n_b = 32$, decay factor $\alpha = 0.9$.
- **SedanSpot**: sample_size = 10000, num_walk = 50, restart_prob = 0.15.
- **Midas-N**: The size of Count-Min Sketches (CMSs) is 2 rows by 1024 columns for all tests.
- **F-FADE**: embedding_size = 200, $W_{upd} = 720$, $T_{th} = 120$, $\alpha = 0.999$, $M = 100$. For t_{setup} , we always use the timestamp value at the 10th percentile of the dataset.
- **MStream**: temporal decay factor $\alpha = 0.95$.
- **DenseAlert**: window=60, to adapt to the data format, we set order = 2.
- **PENminer**: $ws = 1, ms = 1$, view = id, $\alpha = 1, \beta = 1, \gamma = 1$.

E Supplementary Analysis and Results

Table 3: AUC of AnoC on the Wikipedia Dataset under Different Values of b and k

Wikipedia	$k = 0.1$	$k = 1$	$k = 10$	$k = 100$
$b = 0$	0.754	0.754	0.754	0.710
$b = 1$	0.754	0.754	0.754	0.754
$b = 2$	0.710	0.754	0.754	0.754
$b = 3$	0.710	0.754	0.754	0.754

Offset Constant and Decay Coefficient. Table 3 presents the AUC values of AnoC on the Wikipedia dataset with variations in the offset constant b and the decay coefficient k . It is evident that AnoC is insensitive to variations in these two parameters. Intuitively, since the interaction frequency of AnoC is adapted to handle different temporal decays, it is robust to different values of b and k , with the exception of extreme values. In practice, b and k can be selected via testing on small partial datasets, ensuring alignment with specific data characteristics while avoiding the high computational cost of full-dataset tuning.

Table 4: AUC of AnoD on the DARPA Datasets under Different Values of α and β

DARPA	$\beta = 0.1$	$\beta = 0.5$	$\beta = 1.0$	$\beta = 5.0$
$\alpha = 0.1$	0.975	0.974	0.972	0.971
$\alpha = 0.5$	0.968	0.975	0.976	0.970
$\alpha = 1.0$	0.964	0.972	0.975	0.974
$\alpha = 5.0$	0.959	0.970	0.968	0.975

Weight Parameters. Table 4 presents the AUC values of AnoD on the DARPA dataset under different values of α and β . It is observed that AnoD is robust to the weight parameters α and β . This is because in the dataset, the source nodes (initiators) and destination

nodes (recipients) of interaction behaviors play a nearly equivalent role in measuring the anomaly degree. When extending the Interaction Willingness (IW) metric to tasks in other domains, the values of α and β can be effectively determined by leveraging domain prior knowledge or conducting tests on small dataset subsets.

Interpretability Verification. We further evaluate the effectiveness of individual structural components to validate the interpretability of our approach. Specifically, on the DARPA dataset, utilizing solely out-degree or in-degree yields impressive AUC values of 0.968 and 0.955, respectively. These results indicate that the structural scores alone can capture anomalous patterns effectively, providing high-quality, interpretable information. Furthermore, our IW metric, which integrates both dimensions, achieves superior performance (AUC=0.975), demonstrating the value of the combined metric.

Table 5: Dataset Statistics

Dataset	Nodes	Edges	Anomalies	Ratio	Main Type
DARPA	25,525	4,554,344	2,737,209	60.10%	Dense
ISCX-IDS2012	30,917	1,097,070	46,456	4.23%	Dense
CIC-IDS2018	33,176	7,948,748	577,786	7.27%	Dense
CIC-DDoS2019	1,290	20,364,525	20,307,560	99.72%	Dense
Reddit	10,984	672,447	366	0.05%	Counter
Wikipedia	9,227	157,474	217	0.14%	Counter

Table 6: AUC under Different Data Ratios

Ratios	20%	40%	60%	80%	100%
AnoD on DARPA	0.989	0.988	0.988	0.987	0.975
AnoD on ISCX-IDS2012	0.908	0.898	0.972	0.973	0.957
AnoC on Reddit	0.719	0.579	0.592	0.597	0.616
AnoC on Wikipedia	0.728	0.730	0.738	0.743	0.754

Table 7: AUC of AnoD under Different Values of Global Decay Factor γ

γ	DARPA	ISCX-IDS2012	CIC-IDS2018	CIC-DDoS2019
0.2	0.964	0.863	0.855	0.998
0.4	0.967	0.915	0.905	0.999
0.6	0.970	0.939	0.935	0.999
0.8	0.970	0.952	0.968	0.999
1.0	0.888	0.736	0.655	0.980

Table 8: AUC (Running Time) of AnoD (on the DARPA Dataset) and AnoC (on the Wikipedia Dataset) under Different Values of n_r and n_b

AnoD on DARPA	$n_b = 32$	$n_b = 64$	$n_b = 128$
$n_r = 1$	0.973 (8.58s)	0.977 (16.03s)	0.979 (49.27s)
$n_r = 2$	0.975 (15.31s)	0.977 (31.26s)	0.977 (90.37s)
$n_r = 3$	0.979 (21.45s)	0.979 (44.72s)	0.979 (197.74s)
AnoC on Wikipedia	$n_b = 32$	$n_b = 64$	$n_b = 128$
$n_r = 1$	0.552 (3.26s)	0.636 (6.21s)	0.754 (12.82s)
$n_r = 2$	0.636 (6.02s)	0.710 (12.78s)	0.754 (21.40s)
$n_r = 3$	0.636 (10.22s)	0.710 (21.80s)	0.754 (44.35s)